

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 9

Due: Mai 3, 2026

Please note the submission rules at the end of this exercise sheet. Non-adherence to these rules may lead to your submission not being marked.

Exercise 9.1 (1+1 marks)

We investigate phase transitions for arbitrary CNF formulas with n variables, m clauses, and k variables in each clause.

If we vary m while fixing n and k , then large values for m represent an *over-constrained region* since all else being equal having more clauses means having more subformulas connected by a logical and, which constrains the formula more.

For the following variations, do large values for the non-fixed variable lead to an under-constrained or over-constrained region? Justify your answer.

- (a) Varying k while fixing n and m .
- (b) Varying n while fixing m and k .

Exercise 9.2 (2+1 marks)

Consider the following scenario: it is June 24, 2026, the day of the “Foundations of Artificial Intelligence” exam. You just finished studying for the exam together with your friends at the DMI and have a bit of time to spare before the exam. To relax your mind, you want to go swimming in the Rhine and eat an ice cream. Swimming in the Rhine is possible starting at the *Tinquely museum* bringing you back to *Mittlere Brücke* close to the DMI. In the end, you must arrive at the *Organische Chemie* to write the exam. Ice cream can be bought at the *Mittlere Brücke* or close to the *Organische Chemie*.

- (a) Formalize the above scenario as a STRIPS planning task adhering to the following requirements:
 - There are locations *Mittlere Brücke*, *Organische Chemie* and *Tinquely museum*.
 - Initially, you are located at *Mittlere Brücke* (the DMI).
 - Your goal is to be located at *Organische Chemie*, cooled down from swimming in the Rhine and having eaten an ice cream.
 - You can move freely between any pair of locations.
 - You can swim from *Tinquely museum* to *Mittlere Brücke* but not between any other pair of locations.
 - You can enjoy ice cream at *Mittlere Brücke* or at the *Organische Chemie* but not at *Tinquely museum*.
 - Choose a reasonable cost function for the action costs and briefly explain what the costs represent and why they are reasonable in this scenario.
- (b) Provide an optimal plan for your planning task together with its cost.

Exercise 9.3 (0.5+1+1+1.5 marks)

Consider the following variant of the *Seven bridges of Königsberg* problem: for a given set of bridges that cross a river, is there a tour that starts and ends at the same location and crosses each bridge

exactly once? Formally, the problem can be defined as follows: given a graph $G = (V, E)$ with a set of vertices V and a set of edges $E \subseteq V \times V$ and an initial vertex $v_0 \in V$, is there a sequence of vertices from V such that i) all pairs of subsequent vertices are connected by an edge from E , ii) each edge in E occurs exactly once in the sequence, and iii) the first and last vertex of the sequence is v_0 ? For an illustration of the problem, consider the following examples:



The initial vertex is v_0 in both cases. For the graph on the left side, there is such a tour, e.g., $\langle v_0, v_1, v_2, v_0 \rangle$, while there is no such tour for the graph on the right side.

On ADAM, you can find more resources required to solve this exercise in the file `koenigsberg.zip`.

- (a) In `koenigsberg.zip`, you find a PDDL description of the (original instance of the) *Seven bridges of Königsberg* problem. The domain description (variables and actions) is given in the file `bridges-once-domain.pddl`, and the problem description (objects, initial state and goal description) is given in the file `koenigsberg-problem.pddl`. Provide a graphical representation of the problem in the same way as the example above. Please do not forget to mark which location is the initial location.
- (b) Obtain the domain-independent planning system *Fast Downward* from

<https://www.fast-downward.org/latest/releases/24.06/>.

Use *Fast Downward* with a configuration that performs greedy best-first search with the delete relaxation heuristic FF to solve the Seven bridges of Königsberg problem from `koenigsberg.zip`. To do so, invoke the planner with

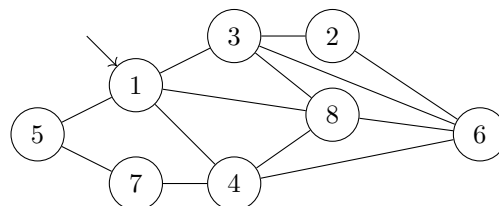
```
./fast-downward.py bridges-once-domain.pddl koenigsberg-problem.pddl
--search "eager_greedy([ff()])"
```

Provide the runtime and the number of expanded states. Discuss whether the solution is optimal. Is the problem solvable? If it is, provide the plan that was found.

- (c) Modify the domain description (`bridges-once-domain.pddl`) such that it is possible to use the same bridge more than once. Solve the resulting problem with the same Fast Downward configuration that was used in part (b) of this exercise.
Provide the runtime and the number of expanded states. Is the problem solvable? If it is, provide the plan that was found.

- (d) Formalize the following instance of the bridges domain in PDDL and solve it with the same Fast Downward configuration that was used in part (b) of this exercise. Use the original domain `bridges-once-domain.pddl`, where each bridge may be traversed only once.

Provide the runtime and the number of expanded states. Discuss whether the solution is optimal. Is the problem solvable? If it is, provide the plan that was found.



Exercise 9.4 (1 marks)

Specify a STRIPS planning task Π which is unsolvable but whose delete relaxation Π^+ is solvable.

Submission rules:

- Submit one solution in groups of up to two students, stating the names of all group members in the submission. All group members are equally responsible for all submitted exercises.
- Create a single A4 PDF file for all non-programming exercises. Solutions in $\text{\LaTeX}$ are preferred, but clearly legible scans in A4 with reasonable contrast and file size are accepted as well.
- In case there are programming exercises, upload a ZIP file (ending .zip, .tar.gz or .tgz; *not* .rar or anything else) including your PDF and those code textfiles required by the exercise. Put your names in a comment on top of each code file. Make sure your code compiles and test it. Code that does not compile or which we cannot successfully execute will not be graded.
- Include a statement on scientific integrity at the end of your submission (see below).

Statement on scientific integrity:

- All submitted solutions must be the product of your own work.
- If your written solutions were produced using any help (e.g., from peers or a large language model), list in what way these people or tools contributed to the solutions.
- We do not mark submissions that lack a statement on scientific integrity.
- Examples:
 - “We solved all exercises on our own.”
 - “We used ChatGPT to explain exercise 7 which we did not understand.”
 - “We had no time to solve this exercise sheet and copied all solutions from our friends.”
(If this is the case, thanks for your honesty but this is plagiarism, results in a warning and no marks. It is preferable to submit no solution at all.)